FieldFlow Business Requirements Doc.

Kurshat Muhammet & Zhuyutong Li

Summary Problem Description:

Field service companies like HVAC, plumbing, electrical, and facilities maintenance continue to rely on outdated methods such as spreadsheets and phone calls to coordinate their technicians. The methods may have been sufficient in simpler times, but they are no longer equipped to handle the complexity of modern day operations. Minor errors in spreadsheets isn't the only problem. Scheduling conflicts arise, skill mismatches occur with lack of real-time insight into technician qualifications, service history becomes unreliable, and the system entirely lacks efficiency as availability is tracked manually.

Beyond internal inefficiencies, clients are left in the dark. With fragmented systems, they're unable to track the status of their service requests or provide structured feedback once a job is complete. Meanwhile, operations managers have no reliable way to measure technician utilization, identify bottlenecks, or make data-driven decisions about workload distribution.

FieldFlow addresses these challenges by providing a structured database system that manages the full lifecycle of a work order. The moment a client submits a service request, through strategized technician dispatch and job execution to final completion and client review, FieldFlow enforces the constraints that manual systems cannot. FieldFlow ensures that the right technician, with the right skills, reaches the right location at the right time.

Business Rules:

A **Client** *submits* one or more **Jobs** describing the work needed.
A **Dispatcher** *dispatches* exactly one **Job** at a time; a **Job** is *dispatched* by one **Dispatcher**.
A **Technician** *is assigned to* zero or more **Jobs**; a **Job** is *assigned to* at most one **Technician**.
A **Technician** *possesses* one or more **Skills**; a **Skill** can be *possessed* by many **Technicians**.
A **Job** *requires* one or more **Skills** through the **Technician-Skill** matching constraint.
A **Job** may *have* zero or one **Review**; a **Review** belongs to exactly one **Job**.
A **Region** *contains* one or more **Technicians**; a **Technician** belongs to exactly one **Region**.
A **Region** *serves* one or more **Clients**; a **Client** belongs to exactly one **Region**.
A **Technician** tracks an availability status and an average rating derived from **Job** reviews.
A **Job** tracks its status (e.g., assigned, in progress, completed), scheduled date, completed date, and priority level.
Client, **Technician**, and **Dispatcher** are all specializations of **Person**, sharing common attributes (name, email, phone)

Identified Actions (possible verbs/relationships):

Client *submits* **Job**
Dispatcher *dispatches* **Job**
Technician *is assigned to* **Job**
Technician *possesses* **Skill**
Job *requires* **Skill**
Job *has* **Review**
Region *contains* **Technician**
Region *serves* **Client**
Client, **Technician**, **Dispatcher** *is a* **Person**

Relational Schema

Primary Keys = <u>underlined</u> // Foreign Keys = *italicized*

Person(<u>personId</u>, name, email, phone)

Client(<u>personId</u>, address, *regionId*)

Dispatcher(<u>personId</u>, department)

Technician(<u>personId</u>, isAvailable, *regionId*)

Region(<u>regionId</u>, regionName, city, state)

Skill(<u>skillId</u>, skillName, category)

TechnicianSkill(<u>technicianId</u>, <u>skillId</u>)

Job(<u>jobId</u>, description, status, scheduledDate, completedDate, priority, *clientId*, *technicianId*, *dispatcherId*)

Review(<u>reviewId</u>, rating, comment, reviewDate, *jobId*)

New Database | Open Database | Write Changes | Revert Changes | Undo | Open Project | Save Project | Attach Database | Close Database

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Create Table | Create Index | Modify Table | Delete Table | Print | Refresh

Name	Type	Schema
Client		CREATE TABLE Client (personId INTEGER PRIMARY KEY, address TEXT, regionId INTEGER NOT NULL, FOREIGN KEY (personId) REFERENCES Person(perso
personId	INTEGER	"personId" INTEGER
address	TEXT	"address" TEXT
regionId	INTEGER	"regionId" INTEGER NOT NULL
Dispatcher		CREATE TABLE Dispatcher (personId INTEGER PRIMARY KEY, department TEXT, FOREIGN KEY (personId) REFERENCES Person(personId))
personId	INTEGER	"personId" INTEGER
department	TEXT	"department" TEXT
Job		CREATE TABLE Job (jobId INTEGER PRIMARY KEY, description TEXT, status TEXT NOT NULL DEFAULT "Assigned", scheduledDate TEXT NOT NULL, compl
Person		CREATE TABLE Person (personId INTEGER PRIMARY KEY, name TEXT NOT NULL, email TEXT NOT NULL, phone TEXT)
personId	INTEGER	"personId" INTEGER
name	TEXT	"name" TEXT NOT NULL
email	TEXT	"email" TEXT NOT NULL
phone	TEXT	"phone" TEXT
Region		CREATE TABLE Region (regionId INTEGER PRIMARY KEY, regionName TEXT NOT NULL, city TEXT NOT NULL, state TEXT NOT NULL)
regionId	INTEGER	"regionId" INTEGER
regionName	TEXT	"regionName" TEXT NOT NULL
city	TEXT	"city" TEXT NOT NULL
state	TEXT	"state" TEXT NOT NULL
Review		CREATE TABLE Review (reviewId INTEGER PRIMARY KEY, rating INTEGER NOT NULL, comment TEXT, reviewDate TEXT NOT NULL, jobId INTEGER NOT NU
reviewId	INTEGER	"reviewId" INTEGER
rating	INTEGER	"rating" INTEGER NOT NULL
comment	TEXT	"comment" TEXT
reviewDate	TEXT	"reviewDate" TEXT NOT NULL
jobId	INTEGER	"jobId" INTEGER NOT NULL
Skill		CREATE TABLE Skill (skillId INTEGER PRIMARY KEY, skillName TEXT NOT NULL, category TEXT)
skillId	INTEGER	"skillId" INTEGER
skillName	TEXT	"skillName" TEXT NOT NULL
category	TEXT	"category" TEXT
Technician		CREATE TABLE Technician (personId INTEGER PRIMARY KEY, isAvailable INTEGER NOT NULL DEFAULT 1, regionId INTEGER NOT NULL, FOREIGN KEY (pers
personId	INTEGER	"personId" INTEGER
isAvailable	INTEGER	"isAvailable" INTEGER NOT NULL DEFAULT 1
regionId	INTEGER	"regionId" INTEGER NOT NULL
TechnicianSkill		CREATE TABLE TechnicianSkill (technicianId INTEGER, skillId INTEGER, PRIMARY KEY (technicianId, skillId), FOREIGN KEY (technicianId) REFERENCES Tech
technicianId	INTEGER	"technicianId" INTEGER
skillId	INTEGER	"skillId" INTEGER
Indices (0)		
Views (0)		
Triggers (0)		

SQL Log

Show SQL submitted by Ap | Clear

```
4   PRAGMA encoding;
5   PRAGMA collation_list;
6   SAVEPOINT "db4s_edittable_1772
7   ROLLBACK TO SAVEPOINT "db4s
8   PRAGMA database_list;
9   SELECT type,name,sql,tbl_name F
10  RELEASE "db4s_edittable_177224
11  SAVEPOINT "UNDOPOINT";
12  SAVEPOINT "RESTOREPOINT";
13  PRAGMA auto_vacuum;
14  PRAGMA automatic_index;
15  PRAGMA checkpoint_fullfsync;
16  PRAGMA foreign_keys;
17  PRAGMA fullfsync;
18  PRAGMA ignore_check_constraint
19  PRAGMA journal_mode;
20  PRAGMA journal_size_limit;
21  PRAGMA locking_mode;
22  PRAGMA max_page_count;
23  PRAGMA page_size;
24  PRAGMA recursive_triggers;
25  PRAGMA secure_delete;
26  PRAGMA synchronous;
27  PRAGMA temp_store;
28  PRAGMA user_version;
29  PRAGMA wal_autocheckpoint;
30  SELECT 'x' NOT LIKE 'X';
31  SAVEPOINT "UNDOPOINT";
32  PRAGMA database_list;
33  SELECT type,name,sql,tbl_name F
34  SAVEPOINT "UNDOPOINT";
35  PRAGMA database_list;
36  SELECT type,name,sql,tbl_name F
37  SAVEPOINT "UNDOPOINT";
38  PRAGMA database_list;
39  SELECT type,name,sql,tbl_name F
40  SAVEPOINT "UNDOPOINT";
41  RELEASE "UNDOPOINT";
42  RELEASE "UNDOPOINT";
43  RELEASE "UNDOPOINT";
44  RELEASE "UNDOPOINT";
45  RELEASE "UNDOPOINT";
46  PRAGMA foreign_keys = ON;
47  SAVEPOINT "UNDOPOINT";
48  SAVEPOINT "RESTOREPOINT";
49  PRAGMA database_list;
50  SELECT type,name,sql,tbl_name F
51
```

UTF-8